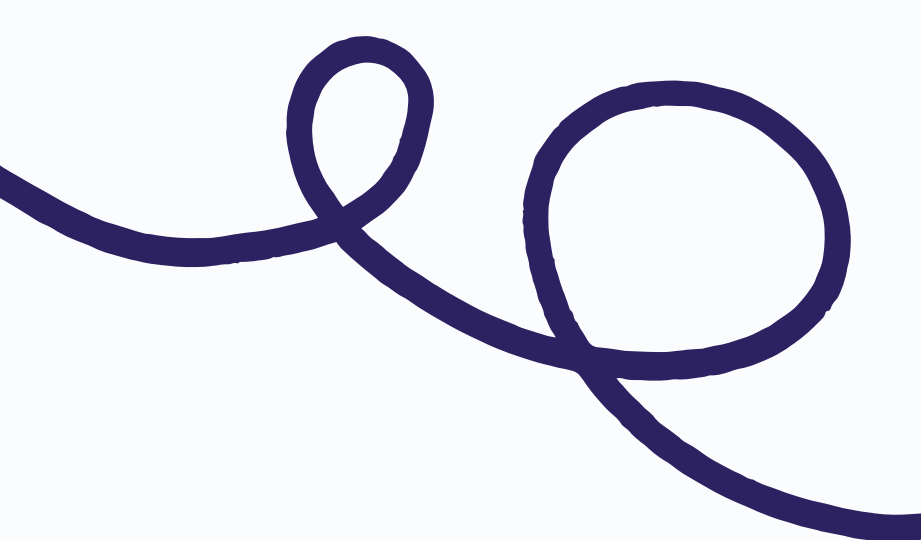




Machine Learning

Analyse comportementale clientèle Retail

| ChurnSight |

présenté par :
Ranim Bouguila



Table de matière

1.Contexte & enjeux

2.présentation de Dataset

3.prétraitement du dataset

4.Clustering k-means

5.classification

6.régression

7.Segmentation

8.conclusion

Contexte & Enjeux

● Problème :

- Acquisition client = 5 à 7× plus cher que la rétention
- Le churn menace la rentabilité à long terme

● Solution :

- **ChurnSight** est une solution **data-driven** qui permet aux entreprises de mieux comprendre, anticiper et optimiser leur relation client.

Objectifs du Projet

3 Tâches ML + Déploiement

Tâche	Question Business	Type de Modèle
◆ Classification	Ce client va-t-il cherner ?	Binaire (0/1)
◆ Régression	Quel montant va-t-il dépenser?	Continue (£)
◆ Clustering	À quel segment appartient-il ?	4 profils RFM
◆ Déploiement	Interface accessible aux métiers	Application Web Flask

Présentation du Dataset

Caractéristiques :

4 372 clients • 52 variables

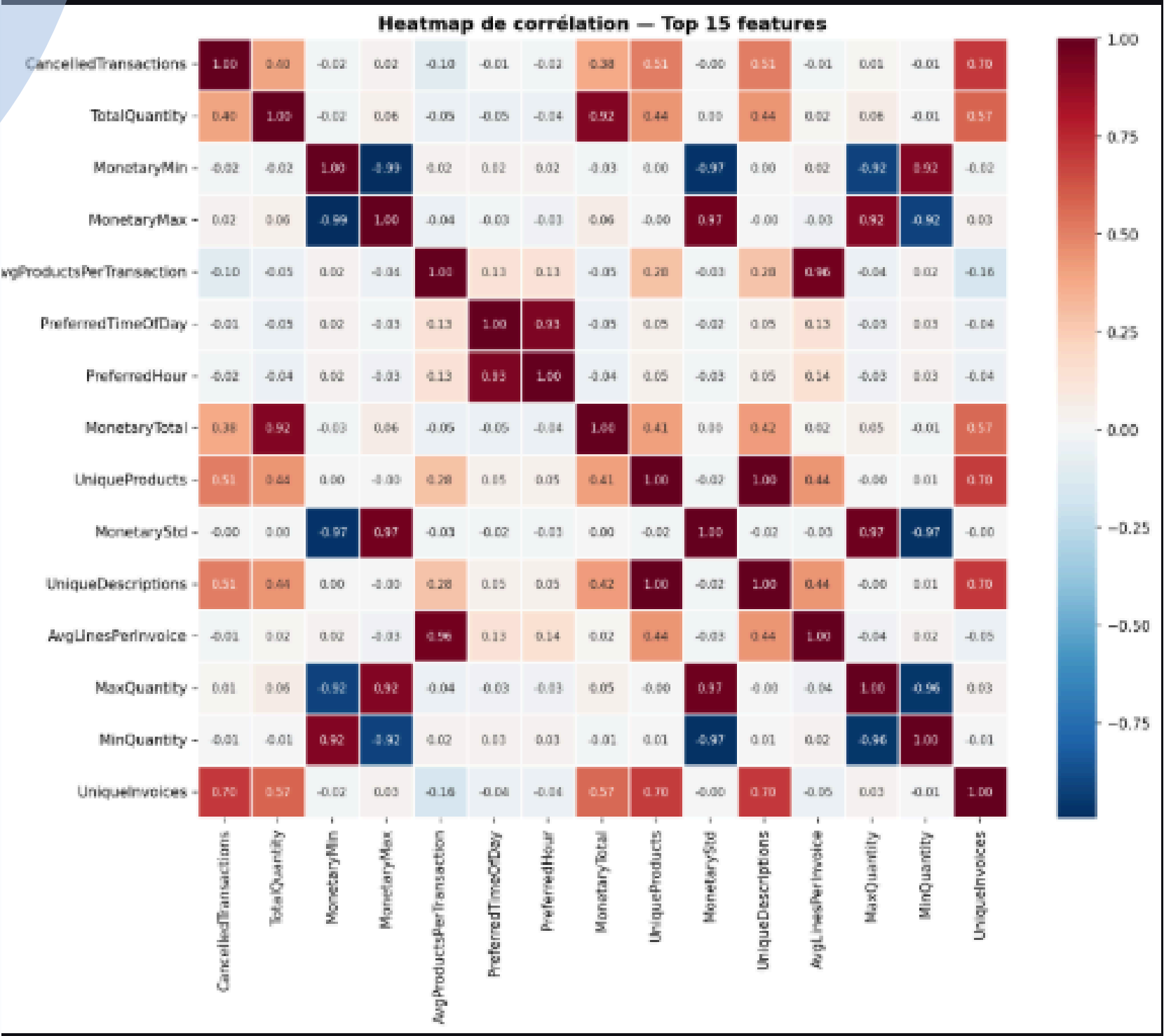
Catégories : RFM, Volumétrie, Comportement temporel, Anomalies, Profil, Engagement

Distribution de Churn :

● Fidèles (0) : 66.7%

● Churnés (1) : 33.3%

Préparation du Dataset



Cette **heatmap** met en évidence une multicolinéarité critique entre plusieurs variables, révélant la nécessité d'une phase de réduction de dimension ou de sélection de variables pour éliminer les redondances avant l'entraînement.

Chargement et exploration initiale

Les données sont chargées depuis un fichier CSV puis analysées afin de :

- identifier les valeurs manquantes
- détecter les doublons
- comprendre la structure des variables

👉 Cette étape permet de poser un diagnostic initial sur la qualité des données.

Nettoyage des données

Plusieurs opérations de nettoyage ont été appliquées :

◆ Suppression des doublons

Les lignes dupliquées ont été supprimées afin d'éviter les biais d'apprentissage.

◆ Imputation des valeurs manquantes

- Variables numériques → imputées par la médiane
- Variables critiques (ex : âge, fréquence d'achat) → méthode robuste aux outliers

◆ Gestion des valeurs aberrantes

- Détection via l'IQR (Interquartile Range)
- Capping des valeurs extrêmes pour éviter qu'elles perturbent les modèles

Traitement des valeurs sentinelles

Certaines variables contiennent des valeurs artificielles (ex : -1, 999).

- 👉 Ces valeurs ont été :
- remplacées par NaN
 - puis imputées par la médiane

Feature Engineering & Encodage

Variables temporelles

À partir de RegistrationDate :

- année
- mois
- jour

jour de la semaine

👉 Permet de capturer des comportements saisonniers.

Analyse des adresses IP

À partir de LastLoginIP :

- décomposition en 4 parties (IP1 à IP4)
- création d'un indicateur IP publique / privée

Stratégies d'encodage :

Méthode	Variables	Justification
Ordinal	AgeCategory, SpendingCategory	Ordre intrinsèque
One-Hot	CustomerType, Region, Gender	Pas d'ordre, faible cardinalité
Frequency Encoding	Country (37 modalités)	Haute cardinalité

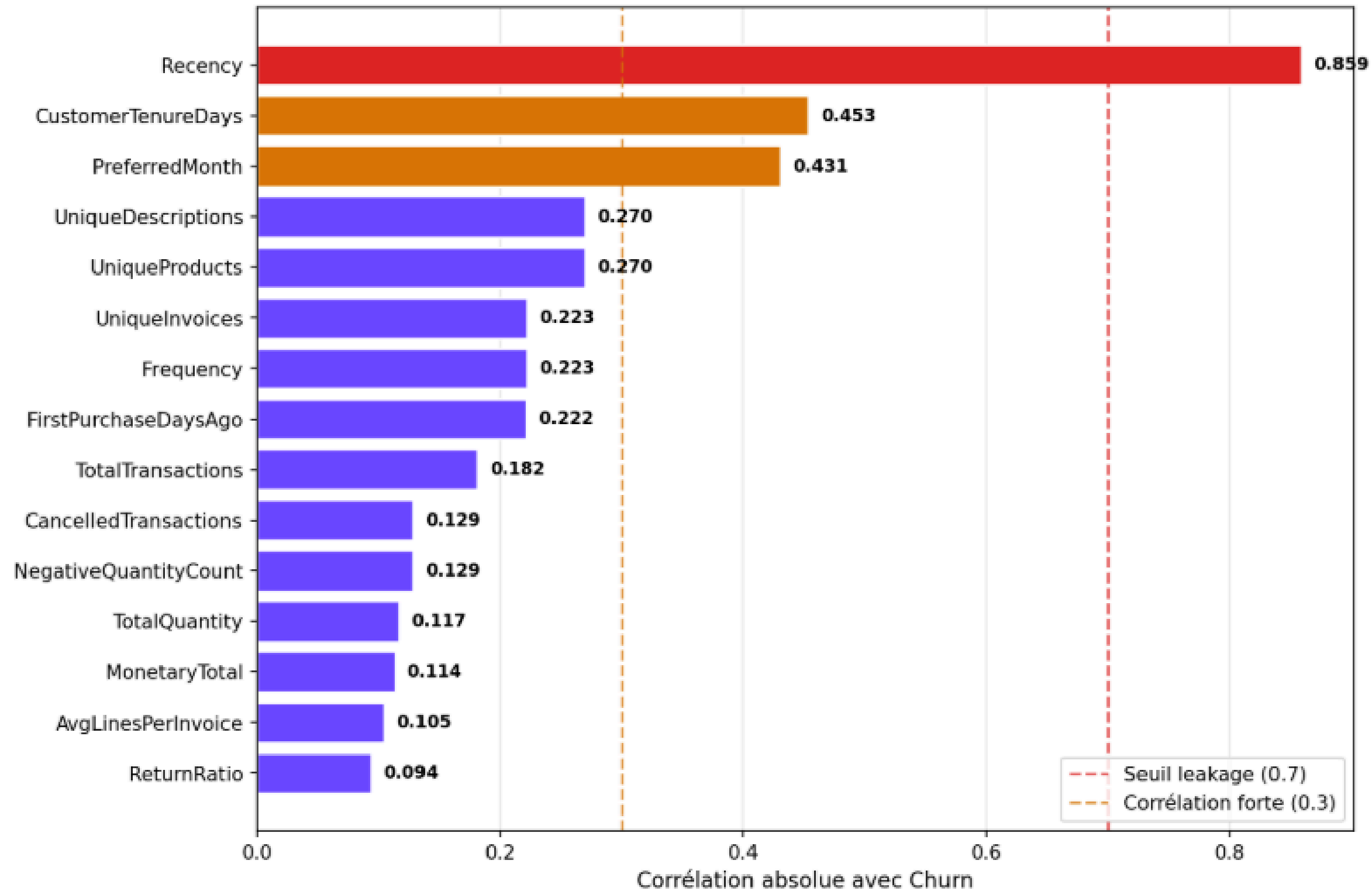
Détection du Data Leakage

Certaines colonnes du dataset sont calculées à partir de la variable cible Churn — elles "connaissent" déjà la réponse avant que le modèle apprenne.

exemple :

ChurnRiskCategory	0.859
Recency	0.71

Top 15 des corrélations avec Churn — Détection du Data Leakage



Détection du Data Leakage

Pourquoi c'est un problème

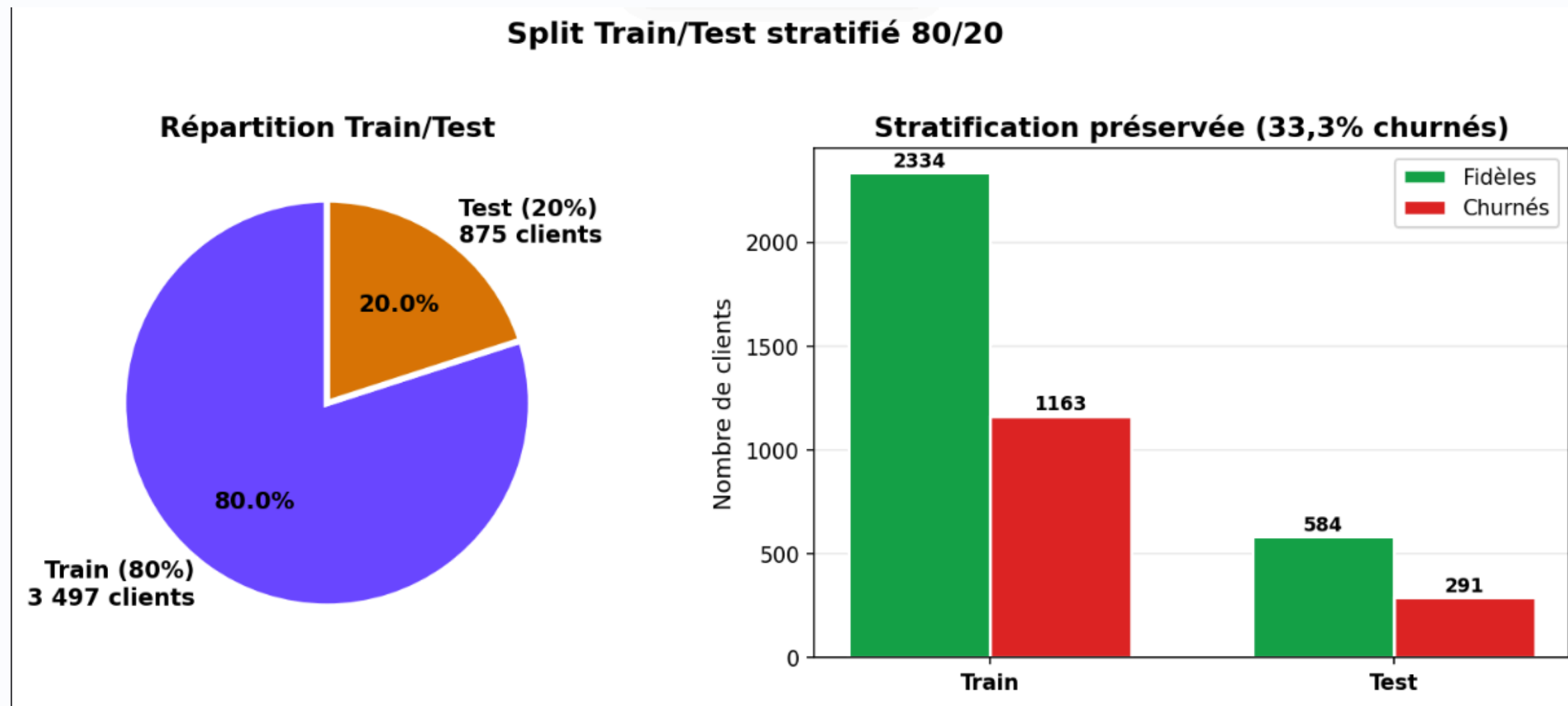
- Résultat sans correction → accuracy 99.9% – score artificiel, inutilisable en production.

Solution appliquée

- Suppression des 6 colonnes leakage avant le split train/test.

Résultat après correction → accuracy 87–96% – score honnête et défendable.

Séparation des données



Le graphique montre un découpage 80/20 équilibré entre train et test, tout en préservant la proportion de clients churnés (33,3 %) dans les deux ensembles.

Normalisation des données

Les variables numériques sont standardisées :

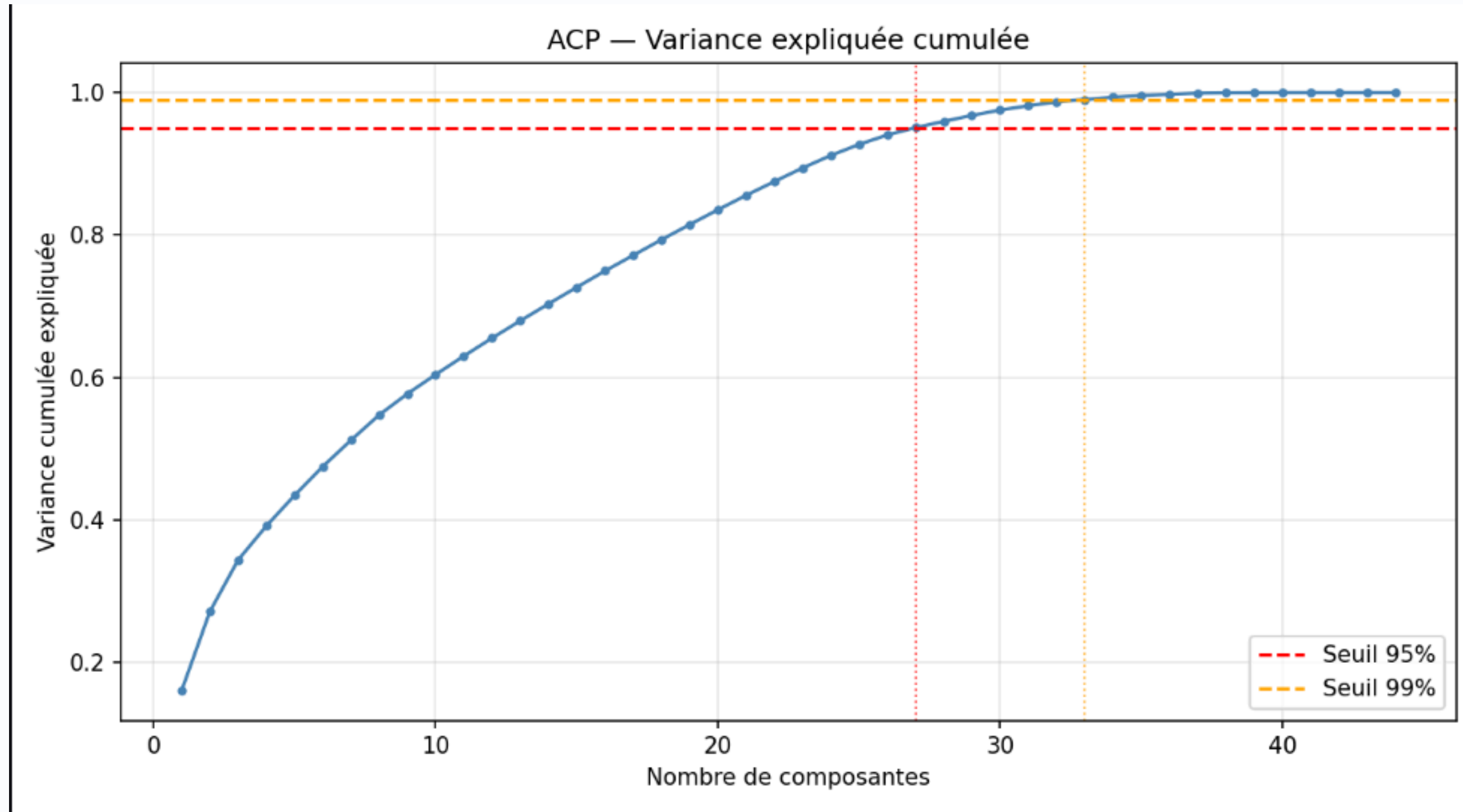
moyenne = 0

écart-type = 1

👉 Utilisation de StandardScaler

⚠ Le scaler est ajusté uniquement sur le train pour éviter toute fuite d'information.

Réduction des dimensions (ACP)



L'ACP montre que 27 composantes suffisent pour conserver 95% de l'information des 44 variables (−38%), mais les variables originales sont gardées pour les modèles arbres.”

10 Gestion du Déséquilibre des Classes

Constat initial

Distribution de la variable cible Churn :

- Classe majoritaire (Fidèles) : 66,7% (2 917 clients)
- Classe minoritaire (Churnés) : 33,3% (1 455 clients)

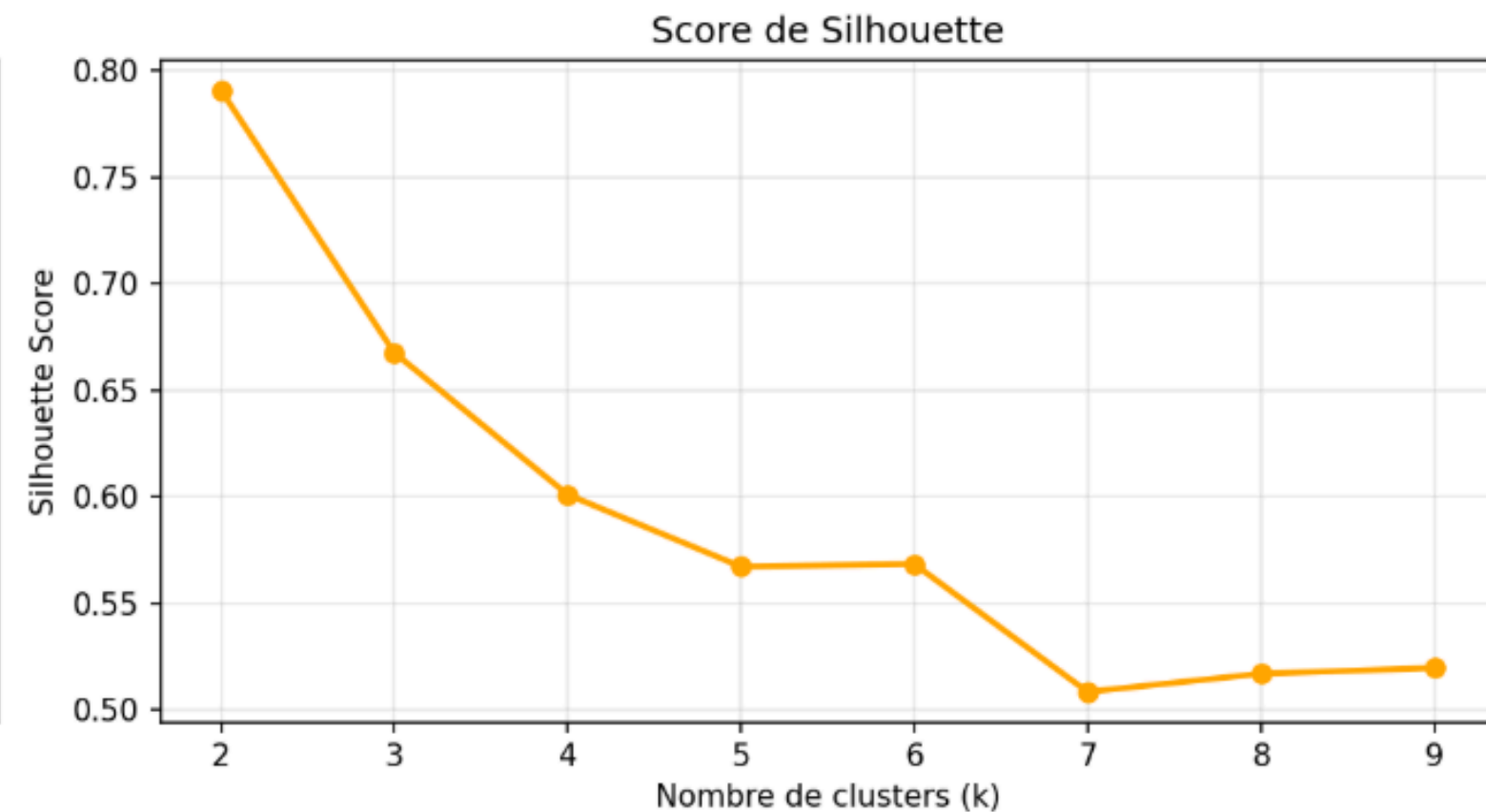
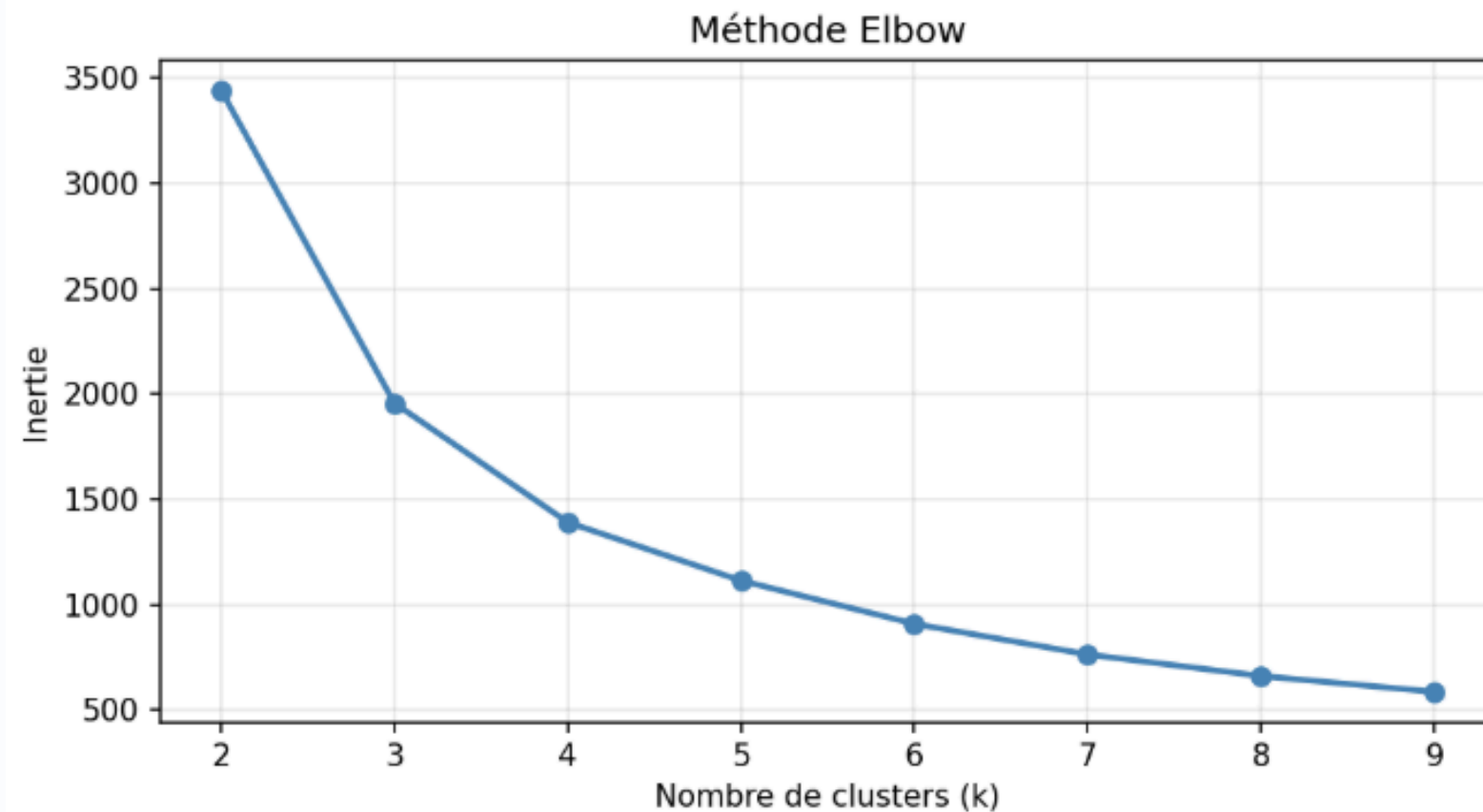
=> Déséquilibre modéré qui nécessite un traitement spécifique pour éviter que le modèle ne favorise systématiquement la classe majoritaire.

Gestion du Déséquilibre des Classes

Stratégies appliquées

Algorithme	Technique	Paramétrage	Objectif	⬇
Random Forest	Pondération des classes	<code>class_weight='balanced'</code>	Donner plus de poids aux churnés	
Logistic Regression	Pondération des classes	<code>class_weight='balanced'</code>	Ajuster automatiquement les poids inverses	
XGBoost	Paramètre scale_pos_weight	<code>scale_pos_weight=2</code>	Accentuer la sensibilité à la classe minoritaire	

Clustering K-Means

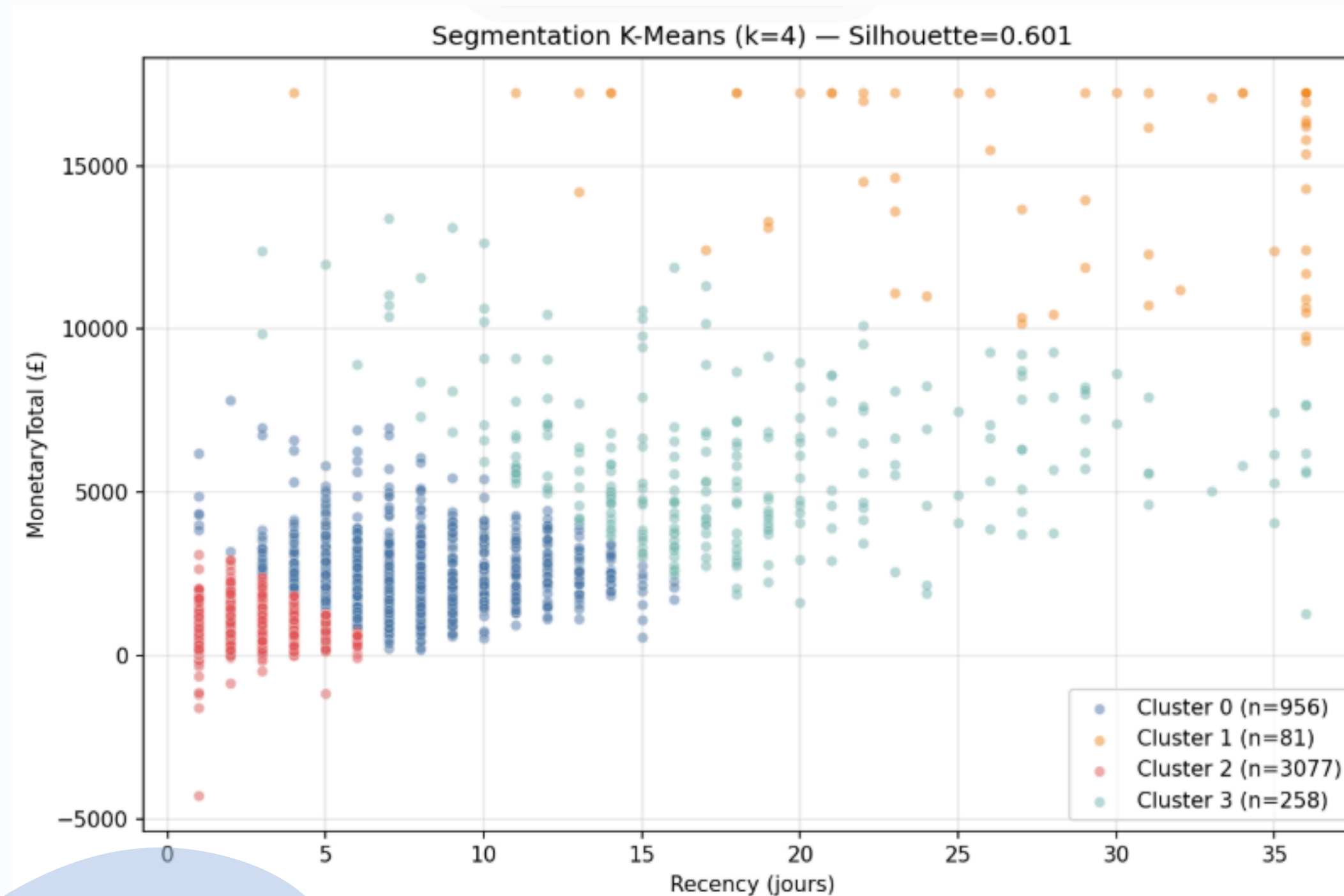


La méthode du coude montre une inflexion à $k=4$ où la réduction d'inertie devient marginale, et le score de silhouette confirme ce choix car au-delà de $k=4$ la qualité des clusters se dégrade progressivement – justifiant le choix de $k=4$ comme nombre optimal de clusters."

Clustering K-Means

Cluster	Effectif	Profil	Action Marketing 
 Dormants	3 337	Faible Frequency & Monetary	Réactivation ciblée
 Réguliers	150	Comportement équilibré	Fidélisation standard
 Champions	6	Très haute activité	Programmes VIP prioritaires
 VIP	4	Dépenses exceptionnelles	Suivi personnalisé 1-to-1

Clustering K-Means

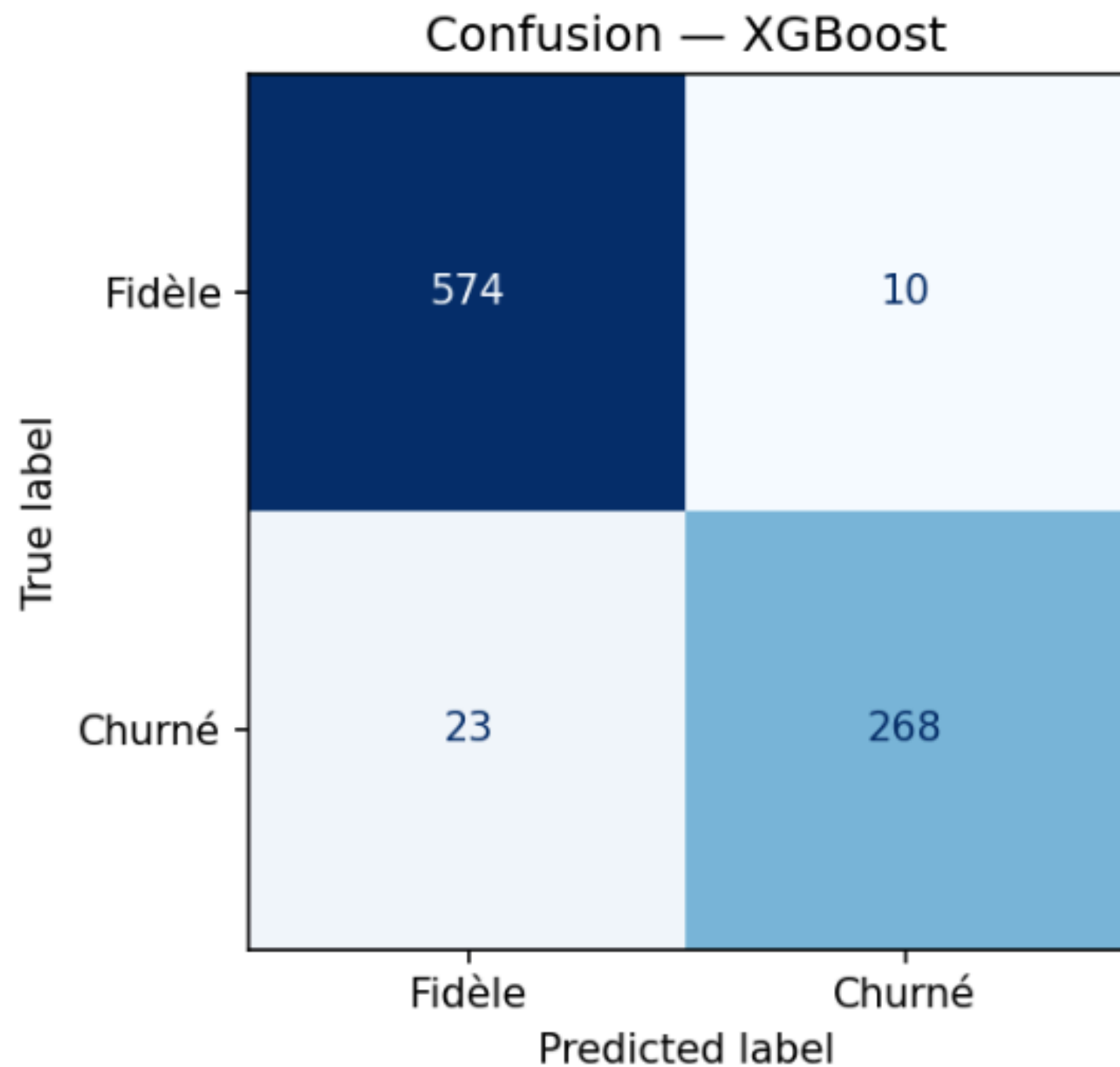


Classification du Churn

Comparaison des algorithmes :

Modèle	Accuracy	F1-Score	Précision	Rappel
Logistic Regression	87.3%	0.875	0.77	0.87
Random Forest	88.7%	0.887	0.83	0.84
XGBoost ★ meilleur	96.2%	0.962	0.96	0.92

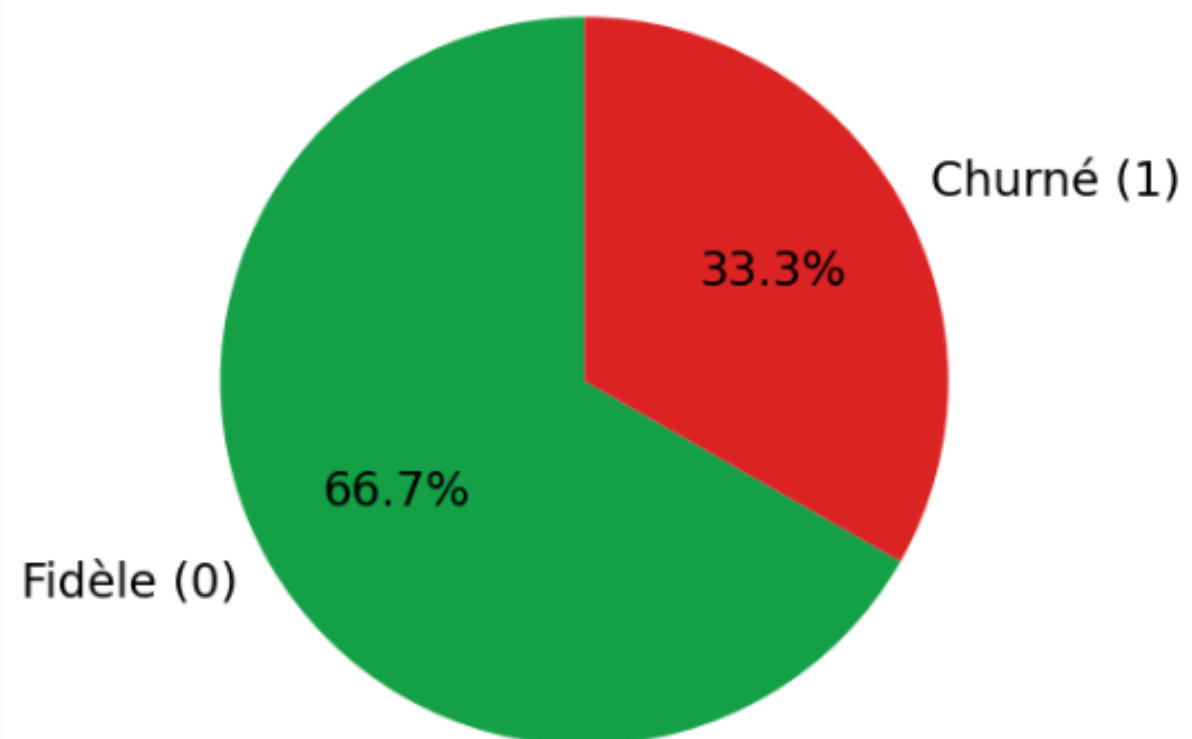
Matrice de confusion (Test set, n=875) :



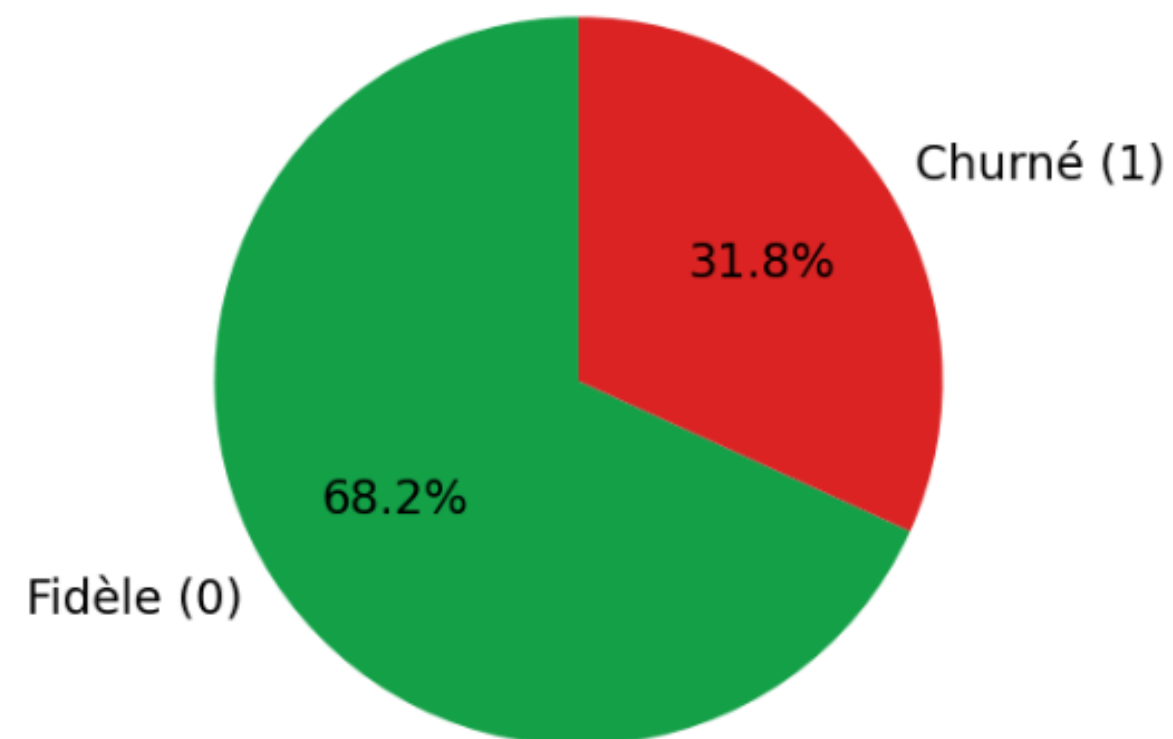
- La matrice de confusion d'**XGBoost** révèle **96,2 % de prédictions** correctes, avec 268 clients à risque correctement identifiés (rappel de 92 %) et seulement 10 fausses alertes sur des clients fidèles.
- Ce faible taux de faux négatifs (23) garantit que la majorité des départs potentiels est détectée à temps pour déclencher des actions de rétention ciblées.

Comparaison Churn Réel vs Prédit

Churn Réel
(Accuracy référence)



Churn Prédit — XGBoost
(Accuracy=96.23%)



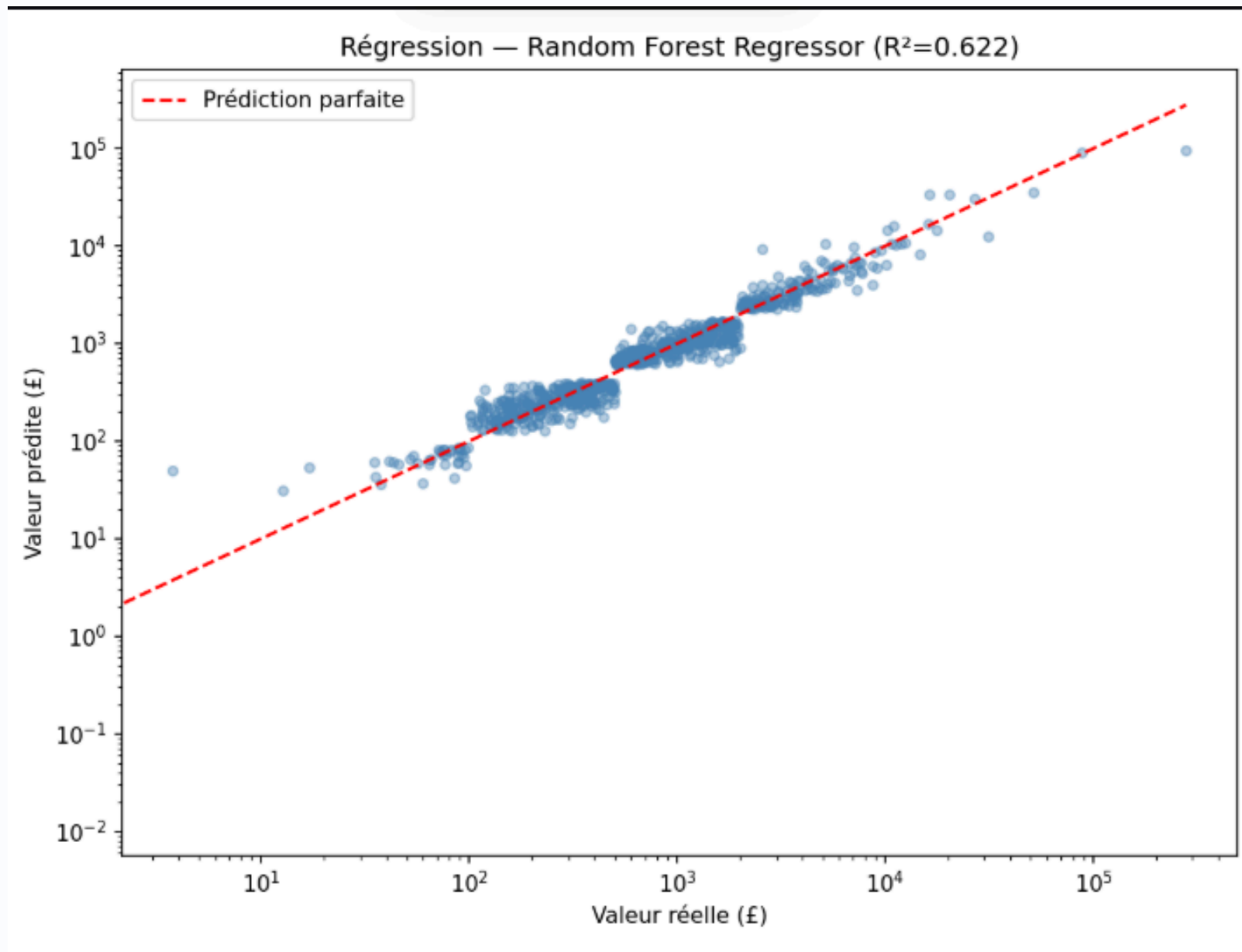
Régression du MonetaryTotal

Prédiction des Dépenses Clients

Modèle	R ²	MAE	RMSE	⬇
Régression Linéaire	0,663	£819	£6 585	
Random Forest Regressor 🏆	0,683	£690	£6 385	
XGBoost Regressor	0,474	£820	£8 226	

Le meilleur modèle est le Random Forest Regressor, qui obtient le R² le plus élevé (0,683) et les erreurs les plus faibles (MAE = £690, RMSE = £6 385), démontrant sa capacité à capturer les non-linéarités du comportement d'achat tout en restant robuste face aux valeurs extrêmes.

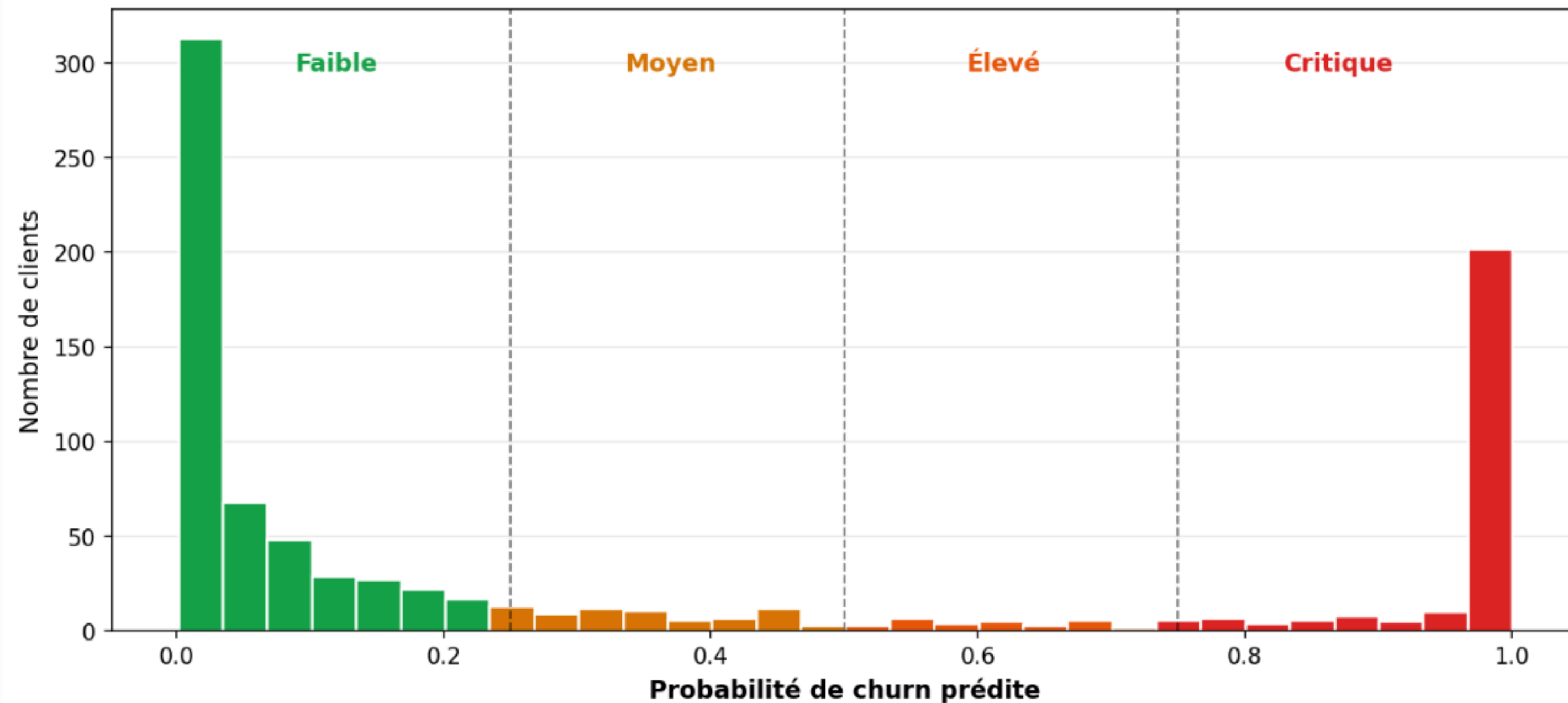
Régression du MonetaryTotal



Voici une visualisation de la performance du modèle Random Forest Regressor, illustrant la corrélation entre les valeurs réelles et les prédictions sur une échelle logarithmique pour évaluer la précision de l'estimation monétaire.

Analyse des Prédiction

Distribution des probabilités de churn (X_test)



Déploiement

Backend : Flask (Python)

- Chargement des modèles .pkl en mémoire
- API REST : GET /, POST /predict, POST /batch

◆ **Frontend : HTML/CSS/JS responsive**

- Sans framework externe
- Utilisable sans compétences techniques

◆ **Expérience utilisateur :**

- Saisie de 10 features métier
- Calcul automatique des features dérivées

FileEditSelectionViewGoRunTerminalHelp

machine-learning-project

EXPLORER

OPEN EDITORS

MACHINE-LEARNING-PROJECT

.ipynb_checkpoints

app

templates

index.html1, D

app.pyM

requirements.txt

data

models

best_classifier.pkl

best_regressor.pkl

clf_features.pkl

kmeans_model.pkl

reg_features.pkl

reg_uses_log.pkl

rfm_features.pkl

scaler_num_cols.pkl

scaler_raw.pkl

scaler_rfm.pkl

scaler.pkl

notebooks

.ipynb_checkpoints

Untitled-checkpoint.ipynb

Untitled.ipynb

reports

src

predict.py

preprocessing.pyM

python.py

Regenerate_figures.py

test.py

train_model.pyM

utils.py

venv

.gitignore

OUTLINE

TIMELINE

index.html1, D

predict.py

test.py

predict_monetary_by_segment.png

predict_proba_distribution.png

app.pyM

app > app.py > compute_derived_features

```
1 # -*- coding: utf-8 -*-
2 from flask import Flask, render_template, request, jsonify
3 import pandas as pd
4 import numpy as np
5 import joblib
6 import os
7
8 app = Flask(__name__)
9
10 # =====
11 # [X] FIX : Chemins avec os.path.join (multi-OS)
12 # =====
13 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
14 MODELS_PATH = os.path.join(BASE_DIR, "models")
15 TRAIN_TEST_PATH = os.path.join(BASE_DIR, "data", "train_test")
16 CLEAN_PATH = os.path.join(BASE_DIR, "data", "processed", "dataset_cleaned.csv")
17
18 print(f"[INFO] BASE_DIR : {BASE_DIR}")
19 print(f"[INFO] MODELS_PATH : {MODELS_PATH}")
20
21 classifier = None
22 regressor = None
23 kmeans = None
24 scaler_raw = None
```

PROBLEMS1

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powershell +

PS C:\Users\ranim\OneDrive\Bureau\machine-learning-project> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\ranim\OneDrive\Bureau\machine-learning-project\venv\Scripts\Activate.ps1)

(venv) PS C:\Users\ranim\OneDrive\Bureau\machine-learning-project> pyth

Conclusion

Le projet ChurnSight couvre une chaîne complète de machine learning allant de l'analyse des données jusqu'au déploiement web, avec trois objectifs principaux : prédiction du churn, estimation de la valeur client et segmentation. Les résultats sont globalement bons, notamment avec un modèle XGBoost performant (accuracy 96,2 %, F1 = 0,962) et une segmentation exploitable. En revanche, la régression reste moyenne ($R^2 = 0,683$) à cause des valeurs extrêmes, et le clustering montre un segment dominant.



merci pour votre attention

lien github: <https://github.com/Ranim638515/machine-learning-project.git>

